YAARALABS PLAYGROUND · NUVAMA ASSET SERVICES AI TRAINING

Experiment 3: Confusion Matrix

Seeing exactly which way a model is wrong, and tuning it for the mistake the business cares about most.

WHY THIS EXPERIMENT

Accuracy alone hides how a model fails; the confusion matrix shows what it caught, what it missed and what it flagged by mistake.

USE CASE FOR NUVAMA

Before trusting a model to triage client messages, Nuvama must know how many urgent ones it would miss, and how many false alarms seniors would chase.

Two ways to start: why there is an "Ex 2" box

1 · MY EX 2 MODEL

- **What:** the Text Recognition model you built in Experiment 2, on your own labelled examples.
- **How:** pick the project and the "class we care about" (e.g. Complaint or escalation vs all other labels); Ex 3 retrains it on 80% of your examples and tests it on the 20% it never saw.
- **Why:** to judge your own model honestly; Ex 2's 100% was measured on data it had already seen.
- **Limit:** with ~36 examples only ~7 are tested, so scores jump in big steps.

2 · A PRE-TRAINED MODEL

- **What:** a model already trained on a 1,600-item sample: Client emails, IT service desk or Customer feedback.
- **How:** pick a dataset and it opens instantly; 320 held-back items are scored.
- **Why:** enough test data for steady scores, and every setting visibly changes the result, so it is the clearest way to learn.
- **Used below:** Client emails.

In short: the Ex 2 box tests **your own** small model; the pre-trained box teaches **how the matrix and settings work** on a large, ready dataset. Both use logistic regression and show the same matrix.

The three sample datasets and where Nuvama would use them

Dataset	What is in it	Where Nuvama uses it	The mistake that matters most
Client emails Escalate vs Routine Used in this walkthrough	Custody client emails about dividends, buybacks, settlements, statements, TDS and fees.	Client-service and custody desks: flag the emails that need a senior person now (missed corporate action, failed settlement, auditor or regulator deadline).	**Missed escalation:** an urgent client waits; a false alarm only costs a senior's time.
IT service desk Incident vs Request	Internal IT tickets on the trading terminal, order system, settlement batch, NAV engine, depository link and VPN.	IT operations: send live outages straight to the on-call team, leave access and software requests in the normal queue.	**Missed incident:** an outage near a cut-off means failed settlements or a late NAV.
Customer feedback Complaint vs Feedback	Investor messages from the app, branches and email about SIP orders, redemptions, contract notes, demat transfers and charges.	Grievance and service teams: separate complaints that must be logged and resolved from suggestions and thanks.	**Missed complaint:** SEBI resolution timelines can be breached and it may escalate to SCORES.

Each dataset has 1,600 items (about one third in the class that matters); the model learns from 1,280 and is tested on 320 it never saw. Below, Client emails is run end to end: 108 Escalate and 212 Routine emails in the test set.

How to read the matrix

Caught (TP)
Real escalations sent to Escalate: what we want.

Missed (FN)
Real escalations left as Routine: the costly mistake, an urgent client waits.

False alarm (FP)
Routine emails sent to Escalate: wastes a senior person's time.

Correct (TN)
Routine emails left as Routine.

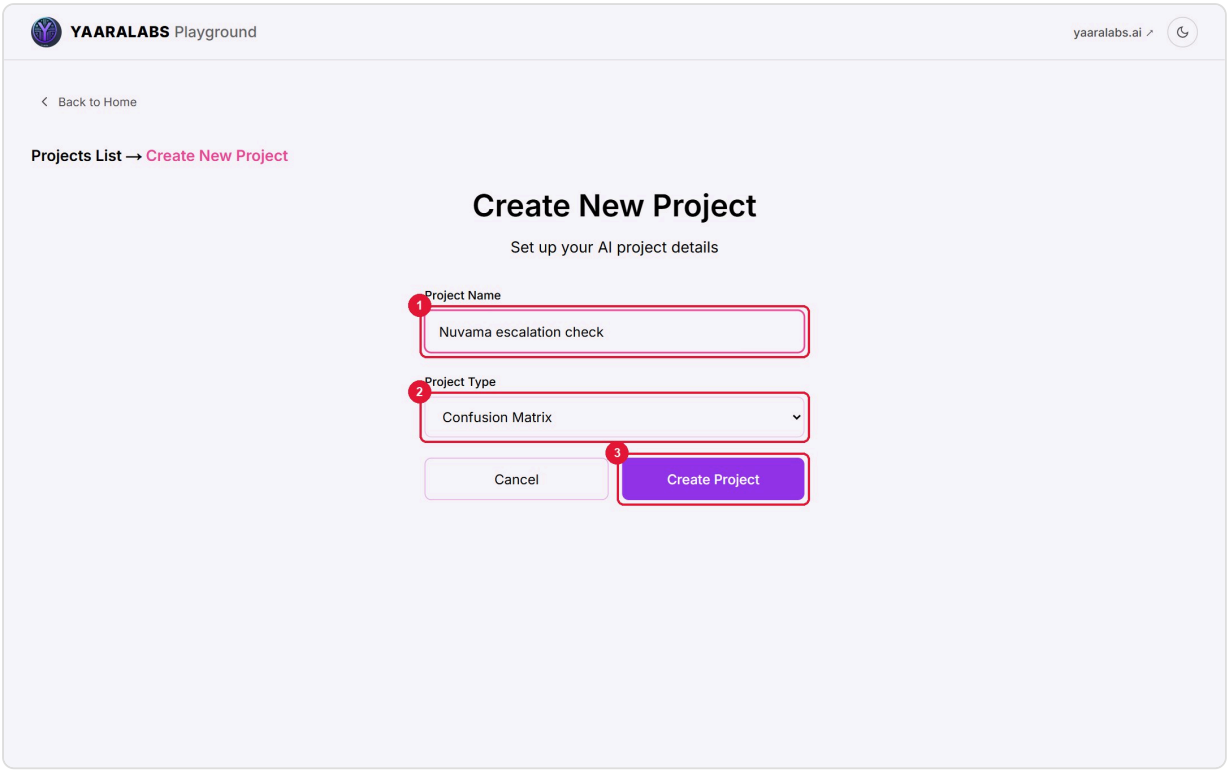
Accuracy = how often it is right overall · **Precision** = when it says Escalate, how often it is right · **Recall** = of all real escalations, how many it caught · **F1** = one score balancing precision and recall.

Step by step

Red boxes mark what is clicked or what to read; numbers show the order.

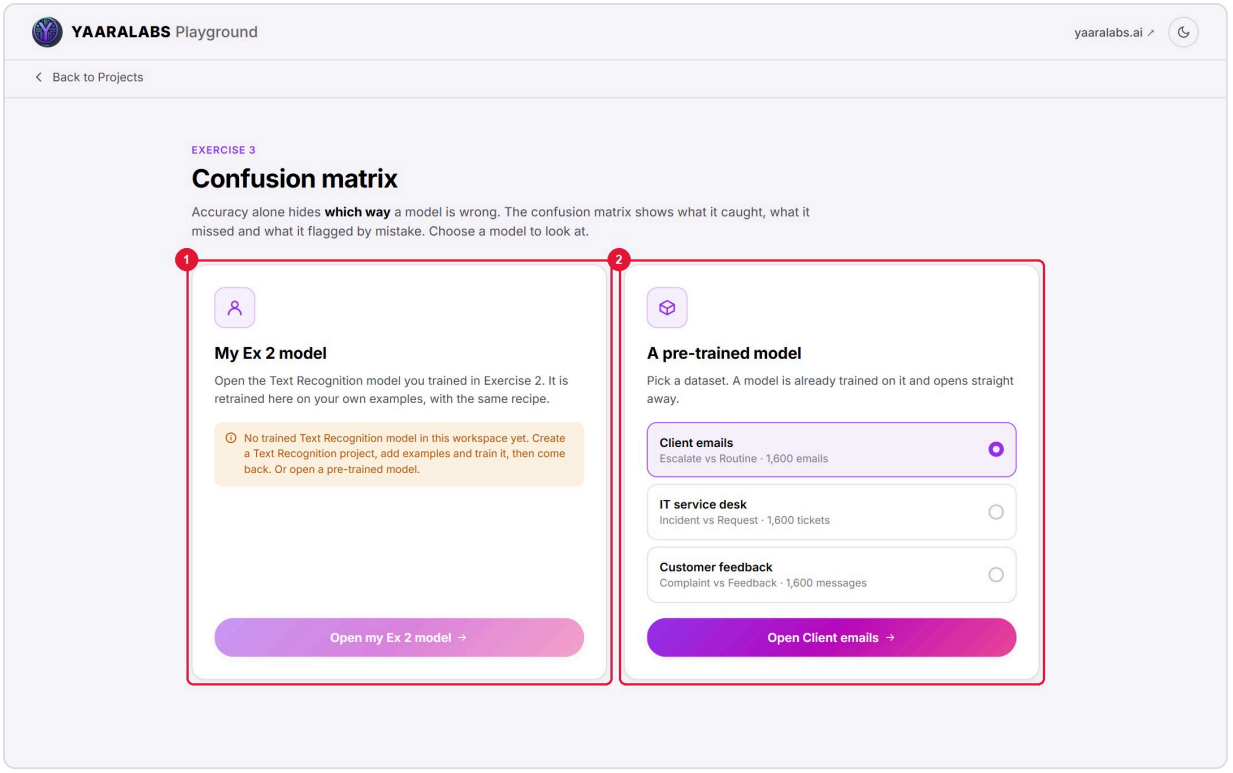
1 Create the project

Type a project name, choose "Confusion Matrix", then click Create Project.



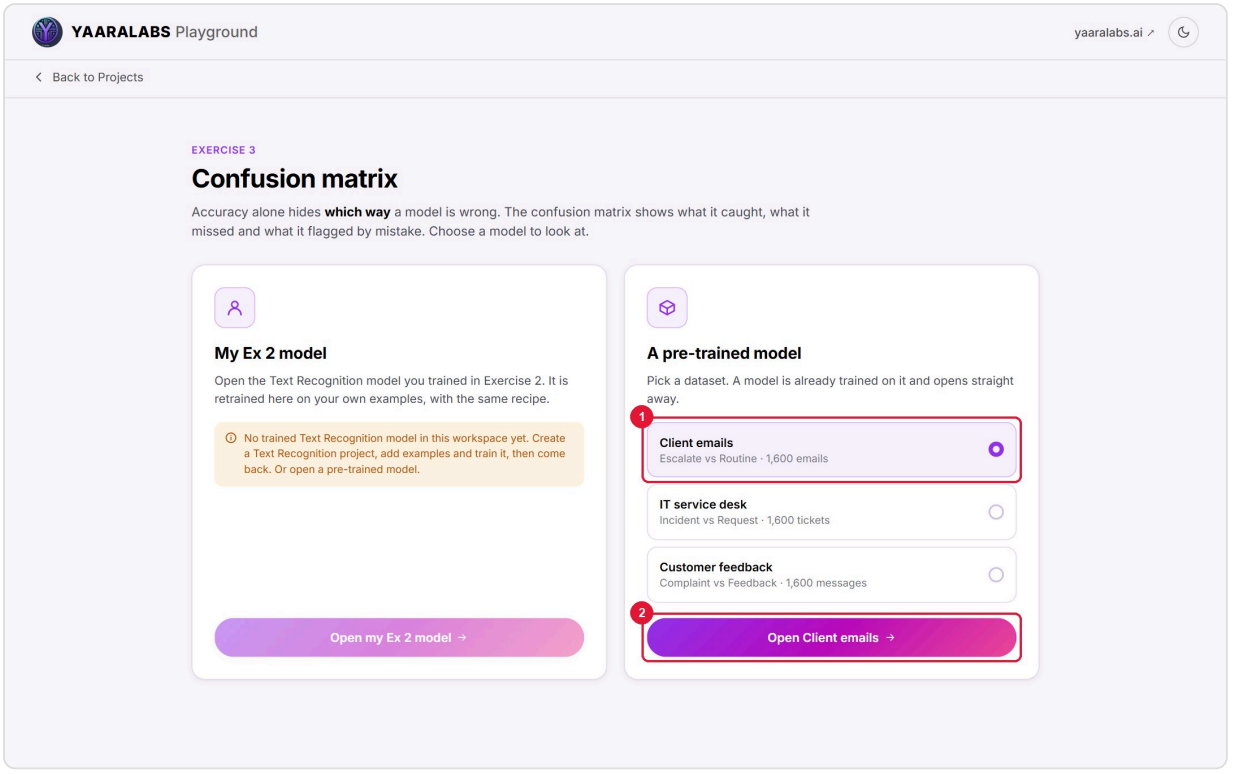
2 Choose where the model comes from

Two ways in: (1) your own model from Experiment 2, or (2) a model already trained on a sample dataset.



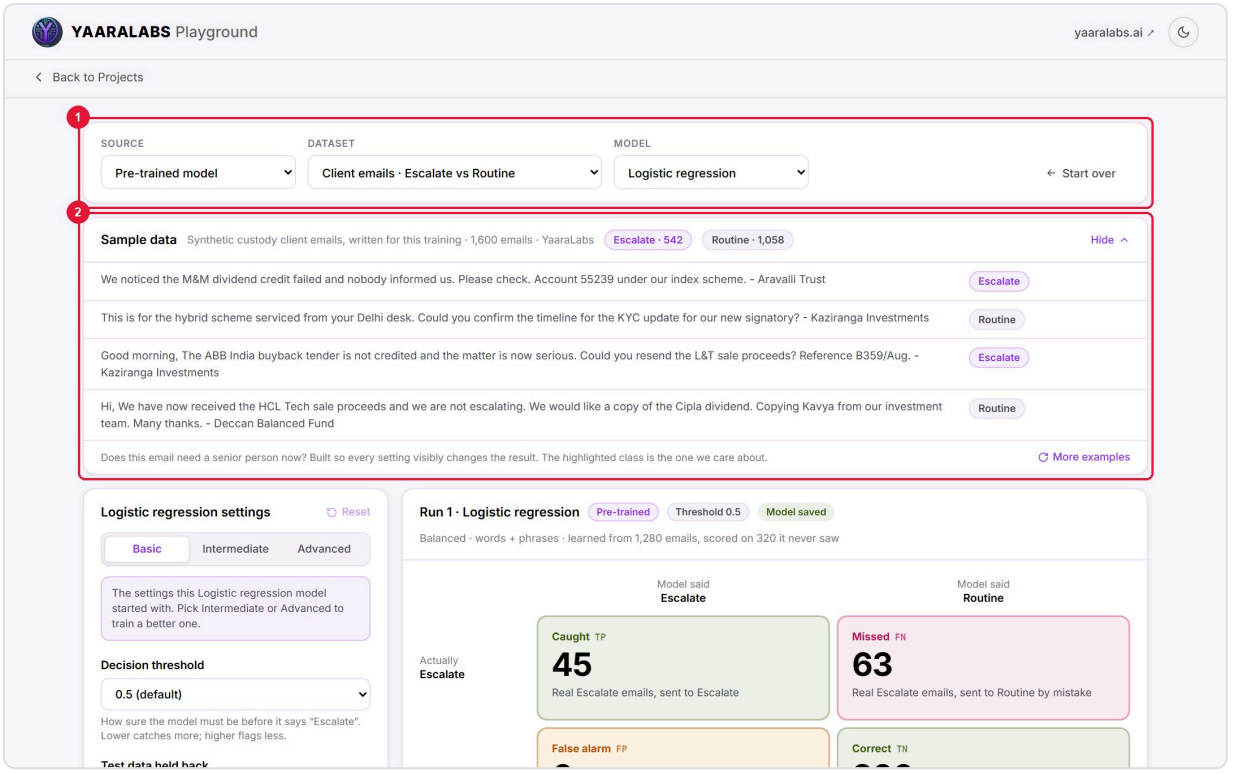
3 Open the Client emails model

Pick the “Client emails” sample (1,600 emails: Escalate vs Routine) and open it.



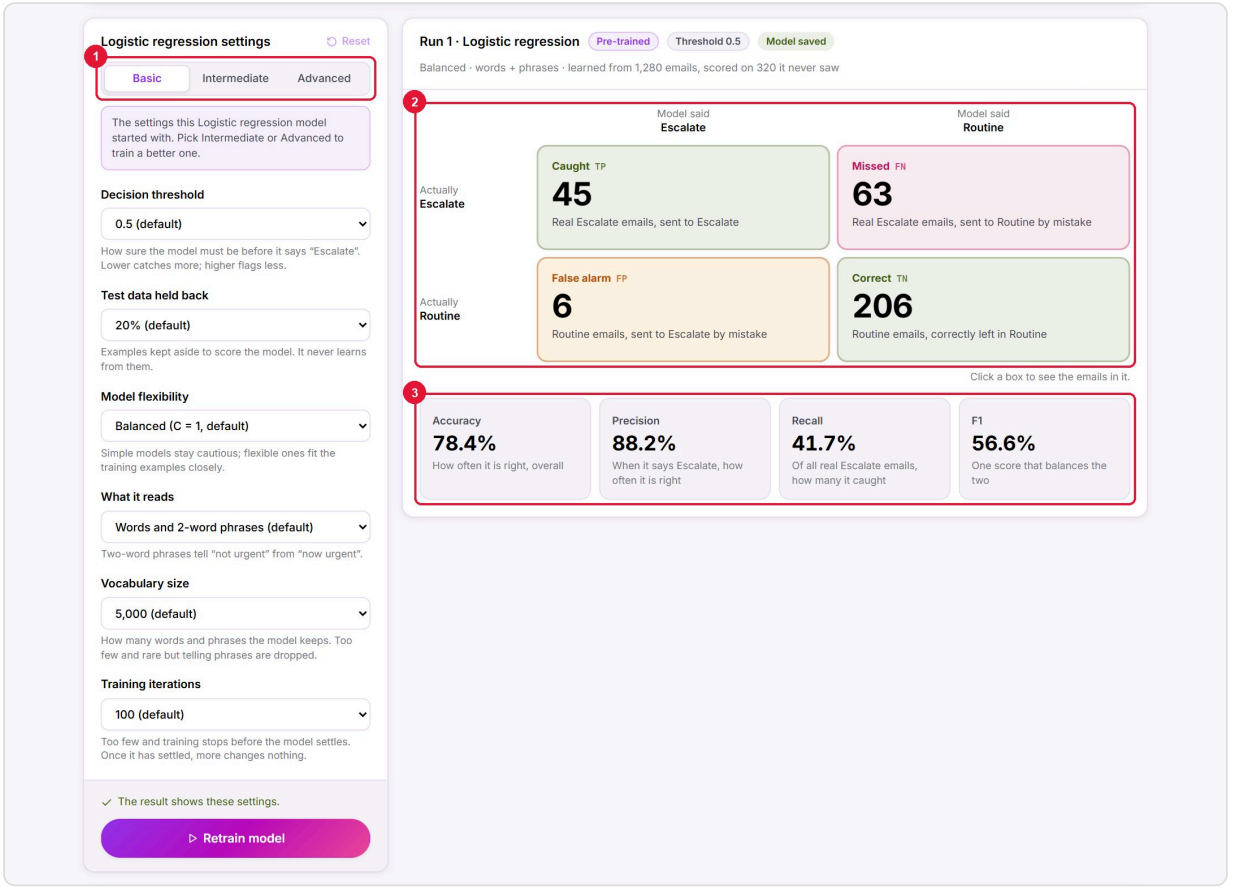
4 The workspace

The workspace shows the source, dataset and model (1), and a few real rows of the data with their true class (2).



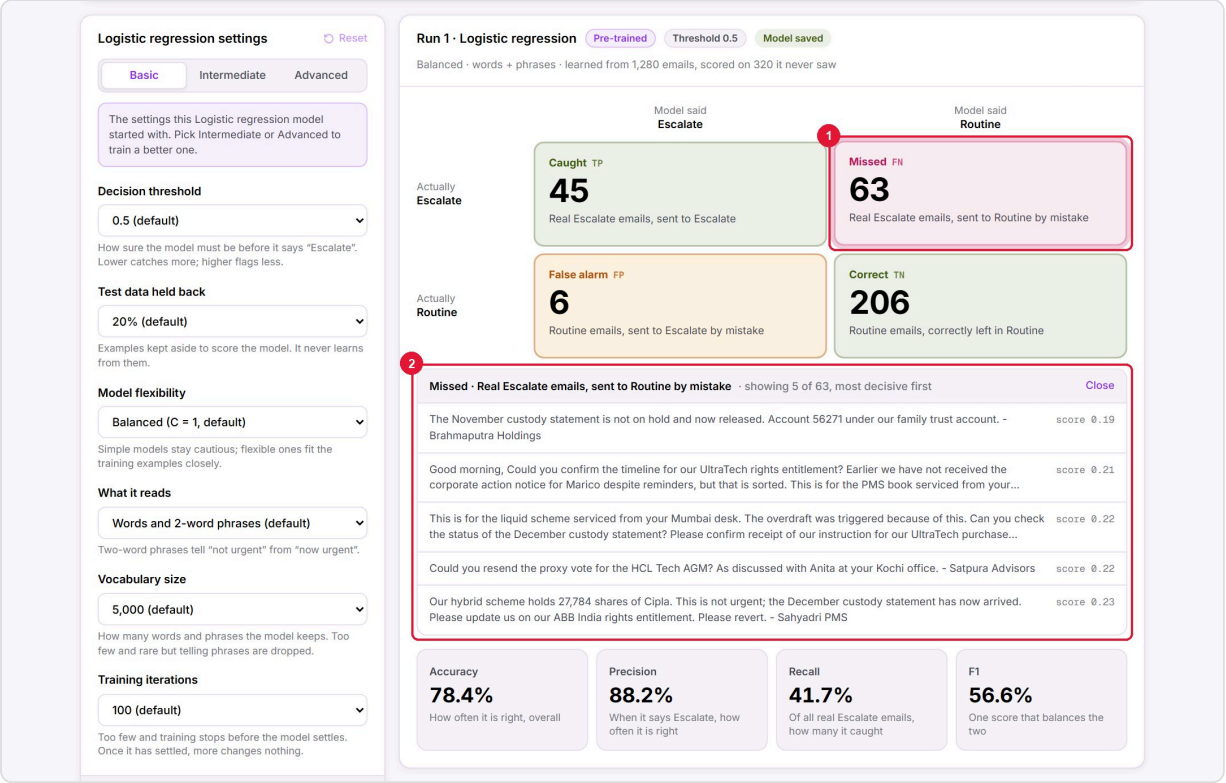
5 Run 1: the confusion matrix

Run 1 is the starting model (Basic). The matrix splits 320 test emails it never saw into four boxes; the four scores sit below.



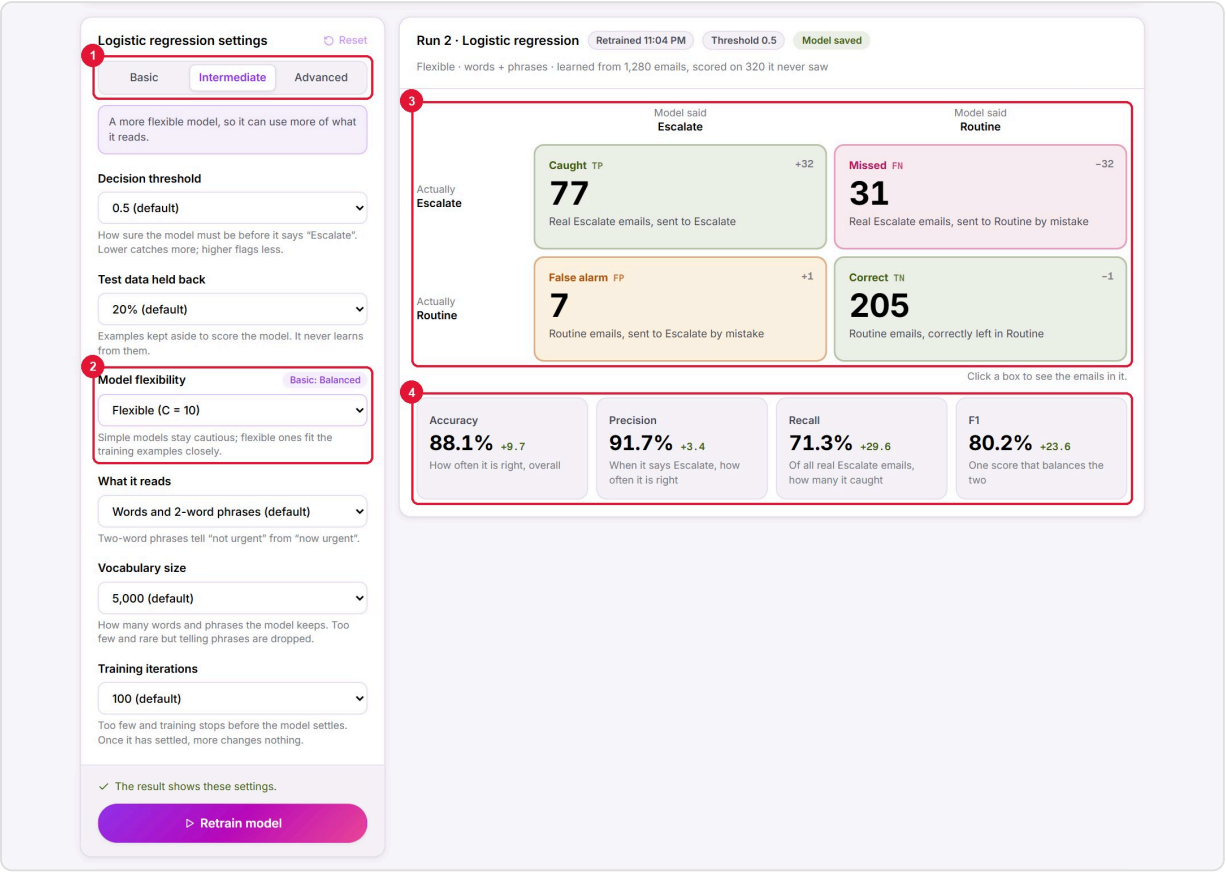
6 Look inside a box

Click a box to read the emails in it. “Missed” lists real escalations the model let through as routine.



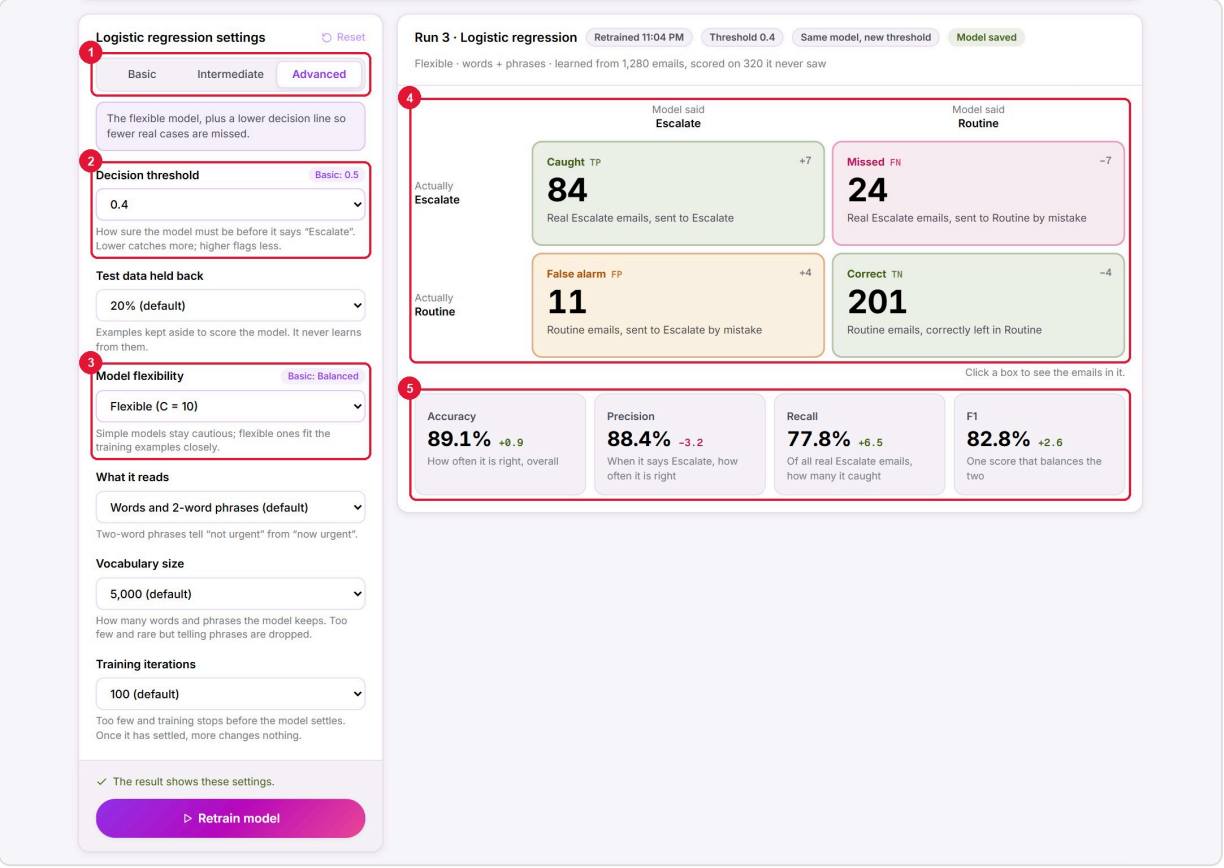
7 Intermediate: a better model in one click

Click Intermediate: the settings change (marked “Basic: …”) and the model retrains at once. Fewer misses, higher scores.



8 Advanced: tune the decision line

Click Advanced: the decision line is also lowered, so even more escalations are caught, at the cost of a few more false alarms.



9 Compare the runs

Every run is kept. Compare runs shows how each change moved Caught, Missed, False alarms and the scores.



Results: three runs on the same 320 test emails

Run	What changed	Caught	Missed	False alarms	Accuracy	Precision	Recall	F1
1	Starting model (Basic): balanced flexibility, threshold 0.5	45	**63**	6	78.4%	88.2%	**41.7%**	56.6%
2	Intermediate: model flexibility raised (C 1 → 10)	77	**31**	7	88.1%	91.7%	**71.3%**	80.2%
3	Advanced: same model, decision threshold lowered 0.5 → 0.4	84	**24**	11	89.1%	88.4%	**77.8%**	82.8%

Conclusion

WHAT THE MATRIX REVEALED

- Run 1 looked fine at 78.4% accuracy, yet missed 63 of 108 real escalations: accuracy hid it.
- A more flexible model (Intermediate) halved the misses.
- Lowering the decision line (Advanced) caught more escalations for a few extra false alarms.

THE BUSINESS DECISION

- For escalations, a miss costs far more than a false alarm, so **recall** matters most.
- The threshold is a business choice: how many false alarms seniors will accept to miss fewer urgent clients.
- Every run is saved, so the choice can be explained and reviewed.

Takeaway: from Run 1 to Run 3, missed escalations fell from 63 to 24 and recall rose from 41.7% to 77.8%, while false alarms grew only from 6 to 11. The confusion matrix turns “is the model good?” into “which mistakes can we live with?”.